ROV Protocols & Controllers
ROV'25 – CERNON

This document aims to make things more clear about what controller we should use and what protocol is best.

ROBOTECH
ROV 2025

PRESENTED BY
ABDRAHMAN EZZAT "Electrical Member"
NOUR AHMED "Electrical Member"

AGENDA

PROTOCOLS

What are the Challenges?

At longer distances, such as 20 meters or more, standard microcontroller communication protocols like UART, SPI, or I2C become inefficient and unreliable for connecting the control station to the ROV. Therefore, a more robust communication method is required. Upon reviewing previous documentation, we found that earlier team members used TTL over UART due to its low cost and ease of implementation. However, this approach led to several issues and proved problematic in practice.

So First, What is TTL UART?

TTL UART is a commonly used serial communication protocol, especially in embedded systems and microcontrollers. It enables two devices to exchange data one bit at a time over two data lines, one for transmitting (Tx) and one for receiving (Rx) along with a shared ground. The "TTL" (Transistor-Transistor Logic) aspect refers to the voltage levels that represent digital signals: typically, 0V for logic 0 and either 3.3V or 5V (depending on the system) for logic 1.

But What problems did it make?

The first issue with TTL UART is its susceptibility to noise over longer distances beyond a few meters (typically around 4 meters), signal integrity begins to degrade. This is a major limitation, especially since our tether may need to support distances of 20 meters or more.

The second problem is its limited data transmission speed. Compared to protocols like SPI and I2C, UART has a significantly lower bit-rate, which could negatively impact the performance of bandwidth-sensitive components like sensors and cameras.

Lastly, UART lacks a built-in system for device addressing or coordination. Unlike I2C, which uses addresses, or SPI, which employs a master/slave configuration with chip select lines, UART doesn’t provide a native way to communicate with multiple devices on a shared bus.

What are the solutions?

After thorough research, we identified two potential alternatives to replace TTL UART: the RS-485 protocol and the Ethernet protocol. To better understand which would be more suitable, we examined official MATE technical reports and noticed that most teams consistently chose Ethernet over RS-485. This led us to question why Ethernet was favorer until we eventually uncovered the reasons behind this preference.

RS-485

RS-485 is a robust and efficient electrical interface for serial data transmission, particularly suited for industrial and building automation applications where reliability over distance and in harsh environments is critical.

Its Main Characteristics are:

Differential Signaling: The key feature of RS-485 is that it transmits data as a voltage difference between two wires (often labeled A and B, or + and -) within a twisted pair cable, rather than a voltage referenced to a common ground. **This "differential" approach makes it highly resistant to electrical noise and allows for long transmission distances.**

Long Distance & Noise Immunity: Because of its differential signaling, RS-485 can reliably transmit data over long distances (up to **1200 meters** or **4000 feet**) and in electrically noisy environments like industrial plants.

Physical Layer Only: It's crucial to understand that RS-485 only defines the electrical aspects (voltage levels, signal timing). It does not specify the communication protocol (like how data is formatted, addressed, or error-checked). Higher-level protocols like Modbus RTU, Profibus, or DMX512 are often built on top of RS-485 to define these rules.

Half-Duplex (typically): RS-485 is typically implemented in a **half-duplex** mode, where data can flow in both directions, but only one device can transmit at a time. This requires careful coordination to manage when each device sends or receives data, in order to prevent collisions. While there is a **4-wire full-duplex** version available, it’s less commonly used. However, we can achieve full-duplex communication by using two separate RS-485 connections and two modules on the controller (in our case, the Arduino) but this is unreliable.

In summary, we found that RS-485 is a powerful protocol with strong noise immunity, an important advantage given that our tether includes network cables running alongside power lines, which can introduce electromagnetic interference (EMI). While RS-485 is more expensive than TTL UART, it remains relatively affordable, especially compared to the higher cost of Ethernet, making it a medium-cost option. However, RS-485 does have its drawbacks. Most notably its limited data rate. This could pose a challenge when transmitting high-bandwidth data such as camera footage and sensor readings, where higher-speed communication is essential.

Ethernet

Ethernet is a **standard for computer networking** that defines how data is transmitted and received over a wired local area network (LAN). It's the most widely used network technology in the world, renowned for its **high speed, flexibility, and scalability**, making it ideal for applications requiring significant data bandwidth and complex networking capabilities.

Its Main Characteristics are:

High Bandwidth (Data Rate): The key feature of Ethernet is its ability to transmit data at very high speeds, typically **100 Megabits per second (Mbps), 1 Gigabit per second (Gbps), or even 10 Gbps and beyond**. This allows for the simultaneous transmission of large volumes of data, such as high-definition video streams, large sensor datasets, and complex control commands, making it indispensable for modern ROVs.

IP-based Network Protocol: Unlike simpler serial communication standards, Ethernet operates as a full **networking protocol** that uses **Internet Protocol (IP)**. This means:

- **Addressing:** Each device on an Ethernet network has a unique IP address, allowing for sophisticated routing and communication between many different devices.
- **Scalability:** You can easily add multiple devices (cameras, sensors, processing units) to the network, and they can all communicate concurrently.
- **Standardized Software Ecosystem:** It leverages common networking protocols (TCP/IP, UDP) and has a vast software development ecosystem, making it easier to integrate with sophisticated control systems and build complex applications.

Full-Duplex Communication: Ethernet inherently supports **full-duplex communication**, meaning that devices can both transmit and receive data **simultaneously**. This significantly increases efficiency and throughput, as there's no waiting for one device to finish transmitting before another can begin.

Complex Hardware and Software Stack: Ethernet requires a more complex hardware and software implementation compared to simpler serial protocols. It involves dedicated Ethernet controllers (NICs - Network Interface Cards), possibly switches and routers, and a robust operating system (like Linux) to manage the network stack. This abstraction allows for powerful applications but increases the complexity of the underlying system.

This comparison shows the difference between RS-485 and Ethernet

Advantage	RS-485	Ethernet
Max Distance	Up to 1200 m	100 m without switches
Noise Immunity	Excellent	Needs shielding
Cost	Low	Higher hardware cost
Speed	Limited	Very high (100 Mbps – 1 Gbps)
Real-time Reliability	Deterministic	Standard Ethernet is non-deterministic
Protocols Support	Not built-in	Many built-in (TCP/IP, HTTP, etc.)
Remote Control	Not easily	Native (IP-based control)
Power Options	Needs extra power lines	Supports PoE
Setup Complexity	Simple (electrically)	Complex (network config needed)
Error Handling	Manual	Built-in (TCP/IP)
Determinism (Control)	Reliable timing	Needs special protocols (e.g., TSN)

1. Distance:

- **RS-485:**
  RS-485 is well-suited for long-distance communication, supporting cable lengths up to 1200 meters under optimal conditions. It's reliable over extended distances without significant signal degradation.
- **Ethernet:**
  Standard Ethernet (with Cat5e or Cat6 cables) supports distances up to 100 meters, and when using high-quality Shielded Twisted Pair (STP) cables, it maintains signal integrity and performance over moderate lengths.

Conclusion:

Distance is not a limiting factor in our application, as we only require a maximum of 50 meters. Both RS-485 and Ethernet comfortably support this range, so distance does not influence the protocol choice in our case.

2. Noise Immunity:

- **RS-485:**
  RS-485 is inherently designed for high noise immunity, making it very reliable in electrically noisy environments like those found in ROV systems. It uses differential signaling over twisted pairs, which helps cancel out electromagnetic interference (EMI), and it remains cost-effective even with long cable runs.
- **Ethernet:**
  Ethernet can also achieve good noise immunity when using Shielded Twisted Pair (STP) cables, which help reduce EMI. However, these shielded cables are typically more expensive than standard RS-485 wiring

  .

Conclusion:
While both RS-485 and Ethernet can handle noise effectively, RS-485 offers strong immunity at a lower cost. Ethernet requires STP cables to match this level of protection, making RS-485 more budget-friendly in terms of noise resistance.

---

3. Cost:

- **RS-485:**
  RS-485 is generally known for its low cost, especially in simple point-to-point or multi-drop configurations. It requires minimal hardware, just transceivers and basic wiring, making it budget-friendly for many control applications.
- **Ethernet:**
  While Ethernet can be expensive in large-scale LANs requiring routers, switches, and complex infrastructure, our use case is much simpler. For the ROV, we only need a basic switch and shielded twisted pair (STP) cable, which keeps the cost relatively low and manageable.

Conclusion:
Although Ethernet is typically more costly in large networks, but in our specific ROV setup, the cost difference is negligible. The minimal equipment needed makes Ethernet a practical and affordable option.

4. Speed & Real-time reliability & Latency:

- **RS-485:**
  RS-485 offers strong real-time reliability in master/slave configurations due to its deterministic nature, If you poll a sensor, you know exactly when you'll get the response. However, its limited bandwidth can lead to delays when transmitting larger volumes of data, reducing its effectiveness in high-speed real-time applications.
- **Ethernet:**
  Ethernet achieves real-time performance through its high bandwidth, allowing rapid transmission of large data streams. While it's not inherently deterministic like RS-485, it delivers low latency suitable for real-time control and video streaming especially when using dedicated point-to-point tethered links. For applications requiring strict timing, industrial Ethernet protocols can be used to enhance determinism.

Conclusion:

For transmitting standard sensor data, both protocols are generally sufficient. However, when it comes to live video streaming or high-speed data transfer, Ethernet clearly outperforms RS-485 due to its greater bandwidth and lower latency. Which means that the Ethernet is faster.

---

5. Protocols Support:

- **RS-485:**
  RS-485 does not natively support communication protocols. Instead, higher-level protocols such as Modbus RTU, Profibus, or DMX512 must be built on top of it to define communication rules, data structures, and addressing.
- **Ethernet:**
  Ethernet natively supports a wide range of communication protocols (such as TCP/IP, UDP, HTTP, MQTT), making it well-suited for complex systems that require standardized, structured, and scalable communication.

Conclusion:

When it comes to protocol support, Ethernet has a clear advantage due to its native compatibility with modern networking protocols, while RS-485 requires additional layers to achieve similar functionality.

6. Remote Control:

- **RS-485:**
  RS-485 is not ideal for transmitting "rich" feedback. If the system requires sending detailed diagnostic information, high-frequency sensor data, or streaming from multiple complex sensors at once, the limited bandwidth of RS-485 will become a constraint.
- **Ethernet:**
  Ethernet enables IP-based communication, allowing the ROV's onboard computer (such as a Raspberry Pi or custom SBC) to have its own IP address. The surface control station can then communicate with it over a standard Ethernet connection.

Conclusion:

For remote control applications, Ethernet is the superior choice due to its higher data capacity and support for IP-based networking.

---

7. Power Options:

- **RS-485:**
  RS-485 does not support power delivery through the communication lines. Separate power wires are required to supply power to remote devices, adding to the complexity of the wiring.
- **Ethernet:**
  Ethernet supports Power over Ethernet (PoE), allowing both data and power to be transmitted over a single cable. This simplifies wiring and reduces the need for additional power lines, especially when using PoE-capable switches and devices.

Conclusion:

Ethernet has a clear advantage in terms of power delivery. While RS-485 requires separate power lines, Ethernet can simplify the system by combining data and power through a single cable using PoE.

8. Setup Complexity:

- **RS-485:**
  RS-485 is electrically simple to set up. It typically involves connecting devices in a daisy-chain or bus topology with minimal configuration, mainly wiring and addressing when using a protocol like Modbus.
- **Ethernet:**
  Ethernet setup can be more complex, especially when dealing with IP addressing, network configuration, and communication protocols. It may require switches, network settings, and possibly some software configuration on both ends.

Conclusion:
RS-485 offers a simpler and more straightforward setup at the hardware level. Ethernet, while more powerful, requires additional effort in terms of network configuration and setup.

---

9. Error Handling:

- **RS-485:**
  RS-485 does not have built-in error handling. Any error detection or correction must be implemented manually at the application level or through higher-level protocols like Modbus RTU, which adds CRC checks but still requires user-side management.
- **Ethernet:**
  Ethernet, especially when using TCP/IP, includes built-in error detection and correction mechanisms. Protocols like TCP automatically handle packet loss, retransmission, and data integrity, reducing the burden on the developer.

Conclusion:
Ethernet has a clear edge in error handling with its built-in support through standard protocols. RS-485 requires manual implementation, making Ethernet more reliable and developer-friendly for complex communication.

10. Determinism (Control):

- **RS-485:**
  RS-485 offers reliable and predictable timing, especially in master/slave configurations. This makes it highly deterministic and well-suited for real-time control tasks where precise timing is critical.
- **Ethernet:**
  Standard Ethernet is not inherently deterministic, as it was designed for general-purpose data networking. To achieve deterministic behavior, special protocols like Time-Sensitive Networking (TSN) or Real-Time Ethernet must be used.

Conclusion:

RS-485 is naturally deterministic and ideal for control-oriented applications. Ethernet can achieve similar reliability, but only with additional protocols, making RS-485 the simpler choice when precise timing is essential.

Final Conclusion: Choosing the Right Protocol for Our ROV

Our investigation into communication protocols for our ROV focused on RS-485 and Ethernet. While RS-485 offers inherent simplicity and good noise immunity, we've determined that its advantages are largely outweighed by Ethernet's capabilities. Ethernet, when paired with Shielded Twisted Pair (STP) cables, can effectively match RS-485's noise immunity while offering a significantly more robust and feature-rich communication solution with more standard protocols and faster bandwidth

Our Optimal Choice:

Considering all factors, our optimal choice for the ROV's communication protocol is **Ethernet**. Its inherent support for a wide range of higher-level protocols, full-duplex operation, sophisticated error handling, superior bandwidth, and higher speeds are critical for the demanding requirements of our ROV.

We plan to establish communication between the surface station and the ROV's controller using Ethernet with TCP/IP. While this will require initial configuration, the long-term benefits in reliability and performance are substantial.

To ensure robust communication, especially given the potential for electromagnetic interference (EMI) within the ROV's tether (which includes power lines), we will utilize **Shielded Twisted Pair (STP) Ethernet cables** for enhanced noise immunity. Furthermore, if we integrate multiple network-enabled devices, such as additional monitors or sensors, a **network switch** will be incorporated into the system to manage and distribute the network traffic efficiently.

CONTROLLERS

Arduino Mega

The Arduino Mega is a powerful microcontroller board based on the ATmega2560. It stands out in the Arduino family due to its significantly larger number of I/O pins, increased memory, and multiple hardware serial ports, making it particularly well-suited for complex projects that require extensive connectivity and processing capabilities.

Its Main Characteristics are:

ATmega2560 Microcontroller: At its core, the Arduino Mega 2560 utilizes the ATmega2560 microcontroller. This powerful chip provides an amount of Flash memory (256 KB, with 8 KB used by the bootloader), SRAM (8 KB), and EEPROM (4 KB), allowing for more elaborate programs and data storage compared to smaller Arduino boards.

Shield Compatibility: The Arduino Mega is designed to be compatible with most shields designed for the Arduino Uno and older Arduino boards. Its layout includes standard pin headers, ensuring a wide range of readily available add-on boards can be used to extend its functionality for specific applications.

Multiple Serial Ports: With four independent hardware serial ports (UARTs), the Arduino Mega can communicate simultaneously with multiple serial devices without relying on software serial implementations, which can be less reliable and consume more processing power. This is a significant advantage for applications requiring communication with various modules like GPS, Bluetooth, or other microcontrollers.

In summary, the Arduino Mega is a highly capable microcontroller board that offers significant advantages for projects demanding extensive I/O, larger memory, and multiple serial communication channels. While it is generally more expensive and physically larger than smaller Arduino boards like the Uno, its expanded resources justify the cost and size for complex applications that would otherwise be constrained by the limitations of less powerful microcontrollers. Its feature set makes it a popular choice for robotics, automation, 3D printing, and other projects…

Raspberry Pi 4

The Raspberry Pi 4 is a revolutionary single-board computer that has significantly blurred the lines between hobbyist microcontrollers and full-fledged desktop PCs. It offers powerful computing experience in a compact and affordable package, making it highly versatile for a wide range of applications, from educational tools to advanced industrial solutions.

Its Main Characteristics are:

Quad-Core ARM Cortex-A72 Processor: At its heart, the Raspberry Pi 4 features a Broadcom BCM2711 System-on-Chip (SoC) with a quad-core ARM Cortex-A72 (ARM v8) 64-bit processor, typically clocked at 1.5 GHz (later revisions can go up to 1.8 GHz). This provides a significant leap in processing power compared to its predecessors, enabling it to handle more demanding tasks and even serve as a light-duty desktop computer.

Enhanced Connectivity: The Raspberry Pi 4 boasts robust connectivity options, including:

- **True Gigabit Ethernet:** Offers significantly faster wired network speeds compared to previous models that were limited by internal USB 2.0 connections.
- **Dual-band Wi-Fi (802.11ac):** Supports both 2.4 GHz and 5.0 GHz frequencies for faster and more reliable wireless networking.

Operating System Flexibility: While primarily designed for Raspberry Pi OS (a Debian-based Linux distribution), the Raspberry Pi 4 can run a variety of other operating systems, including Ubuntu, Windows 10 IoT Core, and various media center distributions, offering significant flexibility for different use cases.

In summary, the Raspberry Pi 4 represents a single-board computing, delivering performance comparable to entry-level desktop PCs. Its powerful processor, extensive RAM options, dual 4K display support, and improved connectivity make it suitable for a wide array of applications, from educational computing and media centers to home automation, light servers, and complex robotics. While its higher processing power means it consumes more power and can generate more heat than simpler microcontrollers like the Arduino, its versatility and capabilities make it an incredibly powerful and cost-effective tool for makers, developers, and educators.

STM32

STM32 is not a single product like the Arduino Mega or Raspberry Pi 4, but rather a vast family of 32-bit microcontrollers (MCUs) and microprocessors (MPUs) designed and manufactured by STMicroelectronics. These powerful and versatile chips are built around various ARM Cortex-M and Cortex-A processor cores, offering an extensive range of performance, power consumption, and peripheral options to suit almost any embedded application.

Its Main Characteristics are:

Diverse ARM Cortex Cores: The core strength of STM32 lies in its adoption of ARM Cortex-M series for microcontrollers and Cortex-A series (A7, A35) for microprocessors. This allows for a broad spectrum of performance, from ultra-low-power, cost-sensitive applications (M0/M0+) to high-performance, real-time control, and digital signal processing (M4/M7 with FPU), and even embedded Linux capabilities (Cortex-A in MPUs).

Scalability and Wide Range of Series: The STM32 family is highly scalable, offering numerous series each tailored for specific needs. This means developers can find an STM32 that precisely matches their project's requirements in terms of processing power, memory (Flash and SRAM), pin count, and integrated peripherals. This "right-sizing" capability often leads to optimized cost and power consumption.

Versatile Applications: Due to their broad range of capabilities, STM32 microcontrollers are found in an incredibly diverse set of applications, including industrial automation (PLCs, motor control, robotics), consumer electronics (smart home devices, wearables, drones), IoT devices, automotive systems, medical equipment, and more.

In summary, STM32 microcontrollers offer a professional-grade solution for embedded system design, characterized by their diverse range of ARM Cortex cores, extensive peripheral integration, high scalability, and strong emphasis on power efficiency. While the learning curve can be steeper than with entry-level platforms like Arduino due to the sheer breadth of options and the need for more low-level configuration, the unparalleled performance-to-price ratio and the powerful development ecosystem make STM32 an industry standard for engineers and advanced hobbyists seeking robust, optimized, and highly customized embedded solutions.

This comparison shows the difference between Arduino Mega, STM32 and Raspberry Pi 4:

Advantage	Arduino Mega	STM32	Raspberry Pi 4
Processing Power & Speed	8-bit AVR @ 16 MHz	32-bit ARM Cortex-M @ up to 180 MHz	Quad-core ARM Cortex-A72 @ 1.5 GHz
Real-time Performance	Moderate (no RTOS, bare-metal control)	Strong (RTOS-capable, real-time interrupt handling)	Weak (not real-time; Linux introduces delays)
I/O Capabilities	Rich in digital/analog pins	High flexibility with many I/O modes	Limited GPIO for embedded control
Enough GPIO Pins	54 digital, 16 analog	Varies by model (often 50+), with multiplexed functions	Around 40 GPIO (some reserved)
Peripheral Interfaces	UART, SPI, I2C, PWM	Multiple UART, SPI, I2C, CAN, ADC, DAC, PWM, timers	USB, HDMI, UART, SPI, I2C, GPIO, Ethernet
Communication Interface	Basic (UART/SPI/I2C)	Strong embedded comms (CAN, UART, SPI, I2C, USB, etc.)	Advanced (Ethernet, USB 3, Bluetooth, WiFi)
Memory (RAM & Flash)	8 KB RAM, 256 KB Flash	Varies (64–512 KB RAM, 256 KB – 2 MB Flash)	2–8 GB RAM, SD card storage
Power Efficiency	Very low power consumption	Very efficient, suitable for battery-powered applications	High power consumption (runs full OS)
Environmental Robustness	Basic (hobby-grade)	Designed for industrial-grade applications	Not designed for harsh environments
Ease of Programming	Very beginner-friendly (Arduino IDE)	Moderate: STM32CubeIDE + HAL/LL, steep learning curve	More complex (Linux-based, requires OS knowledge)
Development Ecosystem	Large community, lots of tutorials/libraries	Strong industrial toolchain, free STM32CubeMX & IDEs	Huge Linux ecosystem, supports full applications

This comparison shows the difference between Arduino Mega, STM32 and Raspberry Pi 4 in the ethernet scale:

Feature	Arduino Mega	Raspberry Pi 4	STM32
Built-in Ethernet Support	No built-in Ethernet	Yes, includes 1 Gbps Ethernet port	Some models include built-in Ethernet MAC (no PHY)
External Ethernet Support	Supported via SPI Ethernet modules (e.g., W5500)	Not required	Requires external PHY (e.g., LAN8720) and connector
Maximum Speed	10 Mbps (W5100) to 100 Mbps (W5500)	1 Gbps	10/100 Mbps (depends on model and PHY used)
Power over Ethernet (PoE)	Not supported natively; requires add-on circuitry	Not natively supported; PoE HAT available	Not supported natively requires external PoE circuit
Protocol Stack Support	Basic TCP/IP via libraries (Ethernet.h)	Full Linux-based networking stack (TCP/IP, HTTP, SSH)	LwIP TCP/IP stack or other embedded stacks (RTOS-based)
Setup Complexity	Moderate; requires wiring and library setup	Minimal; OS handles configuration automatically	Moderate to high; requires hardware config and TCP/IP stack
Typical Use Case Fit	Basic web client/server, telemetry, remote monitoring	High-speed video, web interface, SSH control	Industrial Ethernet, deterministic control, Modbus TCP

Conclusion:

- **Arduino Mega** requires an external Ethernet module such as the W5500. It supports basic Ethernet communication, but is limited in speed and flexibility. Best for simple networking tasks like sending sensor data to a web server.

- **Raspberry Pi 4** has native high-speed Ethernet support, making it ideal for high-bandwidth applications including video streaming, remote desktop access, and complex IP-based communication.

- **STM32** microcontrollers (particularly STM32F4, F7, H7 series) provide Ethernet MAC interfaces but need an external PHY and some configuration. When set up with a TCP/IP stack like LwIP, they are highly capable for real-time and industrial networking applications.

Final Conclusion: Choosing the Right Controller for Our ROV

Our analysis leads us to a clear understanding of the trade-offs involved in selecting a controller for our Remotely Operated Vehicle (ROV).

The **Arduino Mega** is a solid foundation for basic ROV operation. However, we've concluded it's limited to low-level control and won't allow us to implement the advanced features necessary to remain competitive.

The **Raspberry Pi 4** offers significant advantages over the Arduino Mega, particularly in terms of processing power, memory, real-time control capabilities, and native Ethernet compatibility. These strengths are crucial for building a high-performance ROV.

The **STM32** family of microcontrollers sits between the Arduino and Raspberry Pi in terms of processing power. While capable, our team's lack of experience with the ARM architecture (compared to our familiarity with AVR-based projects) presents a steep learning curve that could hinder development.

Our Optimal Choice:

Considering these factors, our optimal choice for the ROV controller is **the Raspberry Pi 4**. Its superior capabilities align best with our need for advanced features and competitive performance.

Should budget constraints prevent us from acquiring the Raspberry Pi 4, our secondary option is **the Arduino Mega**. However, this choice comes with a caveat: if our project requirements grow beyond the ATmega2560's capabilities, **we will then have no alternative but to invest the time and effort into learning and utilizing STM32 boards.**

Resources:

RS-485 Communication Protocol Resources

- https://www.renesas.com/en/document/apn/rs-485-design-guide-application-note
- https://ez.analog.com/ez-blogs/b/engineerzone-spotlight/posts/rs-485-the-communication-protocol-that-keeps-on-giving-part-1
- https://www.sameskydevices.com/blog/rs-485-serial-interface-explained
- https://github.com/Helgayork/serial-port-tools/blob/master/rs485-communication-guide.md

Ethernet Communication Protocol Resources

- https://www.lantronix.com/resources/networking-tutorials/ethernet-tutorial-networking-basics/
- https://www.mouser.com/pdfdocs/Ethernet_Basics_rev2_en.pdf
- https://www.typhoon-hil.com/documentation/typhoon-hil-software-manual/References/ethernet_communication_tcp.html
- https://www.kalatec.com.br/wp-content/uploads/2020/07/Manual-Comunicacao-Ethernet-TCPIP.pdf
- https://github.com/eoghanmc22/mate-rov-2025

Arduino Mega Resources

- https://docs.arduino.cc/hardware/mega-2560/
- https://docs.arduino.cc/hardware/mega-2560/#suggested-libraries
- https://docs.arduino.cc/libraries/ethernet/
- https://repositorio.uisek.edu.ec/bitstream/123456789/2893/7/ArduinoMega2560Datasheet.pdf

Raspberry Pi 4 Resources

- https://www.raspberrypi.com/products/raspberry-pi-4-model-b/
- https://www.raspberrypi.com/documentation/computers/getting-started.html
- https://www.raspberrypi.org/resources/
- https://forums.raspberrypi.com/viewforum.php?f=91&sid=24652016ff58abe2f8321d7c353e7796
- https://vilros.com/pages/raspberry-pi-4-guide-features-specs

STM32 Resources

- https://www.st.com/en/ecosystems/stm32cube.html
- https://www.st.com/content/st_com/en/support/learning/stm32-education.html
- https://community.st.com/
- https://deepbluembedded.com/stm32-arm-programming-tutorials/